CI/CD 개념 정리

> 작성일: 2026-04-01
> 대상: DevOps 입문~중급 학습자
> 범위: CI/CD 전체 개요 — 핵심 개념부터 도구, 아키텍처 패턴, 실무 Best Practice까지

목차

1. CI/CD란?
2. CI — 지속적 통합 (Continuous Integration)
3. CD — 지속적 전달/배포 (Continuous Delivery/Deployment)
4. CI/CD 파이프라인 구조
5. 테스트 전략
6. 배포 전략
7. 브랜치 전략
8. CI/CD 도구 비교
9. GitOps
10. DevSecOps — 보안 통합
11. 컨테이너와 CI/CD
12. DORA 메트릭 — 성과 측정
13. CI/CD Best Practices (2026)
14. 실습 예제 — GitHub Actions 워크플로우
15. 아키텍처 패턴 예시
16. 참고 자료

---

1. CI/CD란?

CI/CD는 소프트웨어 개발 라이프사이클을 자동화하는 핵심 DevOps 방법론이다.
코드를 작성하는 순간부터 사용자에게 전달되는 순간까지의 모든 과정을 **자동화된 파이프라인**으로 연결한다.

전체 흐름: 코드 작성 → 커밋 → 빌드 → 테스트 → 릴리스 → 배포 → 모니터링 → (피드백 루프) → 코드 작성

CI/CD의 세 가지 개념

개념	정의	자동화 범위
CI (Continuous Integration)	코드 변경사항을 주기적으로 통합하고, 자동 빌드/테스트를 수행	코드 → 빌드 → 테스트
CD (Continuous Delivery)	CI 이후, 프로덕션 배포 **직전**까지 자동화. 최종 배포는 수동 승인	코드 → 빌드 → 테스트 → 스테이징 → (수동 승인) → 프로덕션
CD (Continuous Deployment)	모든 단계를 완전 자동화. 테스트 통과 시 자동으로 프로덕션 배포	코드 → 빌드 → 테스트 → 스테이징 → 프로덕션 (전자동)

CI/CD 도입 효과

항목	CI/CD 미적용	CI/CD 적용
빌드/테스트	수동, 비정기적	커밋마다 자동 실행
배포 주기	주/월 단위	일 단위 ~ 수시 배포
버그 발견 시점	배포 후 (비용 ↑)	커밋 직후 (비용 ↓)
롤백	수동, 복잡	자동, 즉시
팀 간 협업	코드 충돌 빈발	작은 단위 통합으로 충돌 최소화

CI/CD가 중요한 이유

- **인적 오류 최소화**: 반복 작업을 자동화하여 실수 방지
- **빠른 피드백**: 코드 문제를 즉시 발견하고 수정
- **일관된 프로세스**: "내 PC에서는 됐는데"를 없앰
- **비즈니스 가치 전달 가속**: 아이디어 → 사용자 전달까지의 시간 단축

---

2. CI — 지속적 통합 (Continuous Integration)

CI의 핵심 원칙

1. **단일 소스 리포지토리**: 모든 코드를 하나의 버전 관리 시스템에서 관리
2. **빈번한 커밋**: 작은 단위로 자주 통합 (하루 수회 이상 권장)
3. **자동 빌드**: 커밋마다 자동으로 빌드 트리거
4. **자동 테스트**: 빌드 후 테스트 자동 실행
5. **빠른 피드백**: 실패 시 즉시 개발자에게 알림
6. **깨진 빌드 즉시 수정**: 빌드 실패 상태를 방치하지 않음

CI 워크플로우

여러 개발자가 Push/PR로 메인 브랜치에 코드를 통합하면, CI 서버가 자동으로 다음 단계를 순차 실행한다.

단계	설명
1. 소스 코드 체크아웃	리포지토리에서 최신 코드를 가져옴
2. 의존성 설치	프로젝트에 필요한 라이브러리/패키지 설치
3. 코드 빌드	소스 코드를 실행 가능한 형태로 컴파일/패키징
4. 유닛 테스트 실행	개별 함수/모듈 단위 테스트 수행
5. 코드 품질 분석	코드 스타일, 복잡도, 중복 검사
6. 보안 스캔	의존성 취약점, 시크릿 노출 여부 검사

결과: **성공** 시 병합 가능 / **실패** 시 담당 개발자에게 알림 발송 후 수정 필요

CI에서 수행하는 주요 작업

단계	설명	대표 도구

단계	설명	대표 도구
코드 체크아웃	리포지토리에서 최신 코드 가져오기	Git
의존성 설치	라이브러리/패키지 설치	npm, pip, Maven, Gradle
빌드	소스 코드를 실행 가능한 형태로 컴파일/패키징	webpack, Docker build, javac
유닛 테스트	개별 함수/모듈 단위 테스트	pytest, JUnit, Jest
코드 품질 분석	코드 스타일, 복잡도, 중복 검사	ESLint, SonarQube, Pylint
보안 스캔	의존성 취약점, 시크릿 노출 검사	Snyk, Trivy, GitLeaks
아티팩트 생성	빌드 결과물(바이너리, Docker 이미지 등) 저장	Docker Hub, ECR, Nexus

---

3. CD — 지속적 전달/배포 (Continuous Delivery/Deployment)

Continuous Delivery vs Continuous Deployment

CI(코드 → 빌드 → 테스트) 단계를 통과한 후, CD 단계에서 스테이징 배포까지는 동일하게 자동으로 진행된다. 이후 프로덕션 배포 방식에서 두 가지로 나뉜다.

- **Continuous Delivery**: 수동 승인(Manual Approval) 후 프로덕션 배포
- **Continuous Deployment**: 자동으로 프로덕션 배포

구분	Continuous Delivery	Continuous Deployment
프로덕션 배포	수동 승인(Manual Approval) 필요	완전 자동
적합한 경우	규제 환경, 보수적 릴리스 정책	높은 테스트 커버리지, 성숙한 모니터링
신뢰 수준	중간 — 사람이 최종 판단	높음 — 파이프라인에 완전한 신뢰
배포 빈도	주 수회 ~ 일 1회	일 수회 ~ 수십 회

CD 워크플로우

단계	설명
1. 아티팩트 저장소 등록	CI 통과 후, 빌드 결과물을 저장소에 등록 ("Build Once, Deploy Anywhere")
2. 스테이징 환경 배포	프로덕션과 동일한 환경에 배포 후 통합/E2E 테스트 실행
3. 프로덕션 배포	자동 또는 수동 승인 후 배포, 헬스 체크 및 모니터링 수행
4. 결과 처리	정상이면 완료 / 이상 감지 시 자동 롤백

흐름: CI 통과 → 아티팩트 저장 → 스테이징 배포 + 통합/E2E 테스트 → 프로덕션 배포 + 헬스 체크 → 정상(완료) or 이상(자동 롤백)

핵심 원칙: Build Once, Deploy Anywhere

- CI에서 **아티팩트를 한 번만 빌드**하고, 동일한 아티팩트를 dev → staging → production에 배포
- 환경별 차이는 **환경 변수**나 **설정 파일**로 관리

- 환경마다 다시 빌드하면 불일치 위험 발생

---

4. CI/CD 파이프라인 구조

파이프라인의 기본 구성 요소

요소	설명
트리거 (Trigger)	파이프라인을 시작하는 이벤트. Push, PR, 스케줄, 수동 실행 등
스테이지 (Stage)	파이프라인의 논리적 단계. Build, Test, Deploy 등
잡 (Job)	스테이지 내에서 실행되는 작업 단위. 병렬 실행 가능
스텝 (Step)	잡 내의 개별 명령어/액션
아티팩트 (Artifact)	파이프라인에서 생성되는 결과물 (바이너리, 이미지, 보고서 등)
환경 (Environment)	배포 대상 환경 (dev, staging, production)
게이트 (Gate/Approval)	다음 스테이지 진행 전 품질/승인 조건

전형적인 파이프라인 흐름

스테이지	포함 작업	비고
Source	Git Clone	소스 코드 가져오기
Build	Compile, Package, Image Build	실행 가능한 아티팩트 생성
Test	Unit, Integration, Lint, Security Scan	품질 및 보안 검증
Stage	Deploy + E2E Test	프로덕션 동일 환경에서 검증
Prod	Deploy + Monitoring	실제 서비스 배포 및 모니터링

흐름: Source → Build → Test → Stage → (Manual Gate) → Prod

> **Manual Gate**: Continuous Delivery 모델에서는 Stage와 Prod 사이에 수동 승인 게이트가 위치한다.

파이프라인 성능 목표

지표	목표	설명
전체 파이프라인 실행 시간	**15분 이내**	개발자 피드백 루프 유지
유닛 테스트	**5분 이내**	병렬 실행으로 단축
빌드	**5분 이내**	캐싱, 멀티스테이지 빌드 활용
배포	**5분 이내**	자동화된 배포 전략 사용

---

5. 테스트 전략

테스트 피라미드 (Testing Pyramid)

테스트의 종류와 비율을 구조화한 모델. 아래로 갈수록 빠르고 저렴하며, 위로 갈수록 느리고 비싸다.

계층	비율	설명	특징
E2E 테스트 (최상위)	~10%	실제 사용자 시나리오 검증	느림, 비용 높음
통합 테스트 (중간)	~20%	컴포넌트 간 상호작용 검증	보통 속도, 중간 비용
유닛 테스트 (최하위)	~70%	개별 함수/모듈 검증	빠름, 비용 낮음

테스트 유형 상세

유형	대상	속도	CI/CD 실행 시점	대표 도구
유닛 테스트	개별 함수/클래스	매우 빠름	모든 커밋	pytest, JUnit, Jest, Go test
통합 테스트	모듈 간 상호작용, API 계약	보통	모든 커밋 / PR	Testcontainers, Supertest
E2E 테스트	전체 사용자 워크플로우	느림	PR 병합 전 / 릴리스 전	Playwright, Cypress, Selenium
성능 테스트	부하, 응답 시간	느림	릴리스 전 / 스케줄	k6, Locust, JMeter
보안 테스트	취약점, 의존성	보통	모든 커밋	Snyk, Trivy, OWASP ZAP
스모크 테스트	핵심 기능 동작 여부	빠름	배포 직후	curl, 커스텀 스크립트

테스트 실행 전략

커밋 Push 이후 아래 순서로 테스트가 실행되며, 각 단계에서 실패 시 즉시 중단(Fail-Fast)한다.

1. 유닛 테스트 (병렬 실행, ~2분) → 실패 시 즉시 중단
2. 통합 테스트 (병렬 실행, ~5분) → 실패 시 즉시 중단
3. E2E 테스트 (핵심 시나리오만, ~8분)
4. 보안 스캔 (의존성 + 코드)
5. 모든 테스트 통과 → 배포 가능

> **Flaky Test (불안정한 테스트)**: 동일 코드에서 성공/실패가 무작위로 발생하는 테스트. 파이프라인 신뢰도를 떨어뜨리므로 **격리(quarantine)** 후 별도 수정해야 한다.

---

6. 배포 전략

주요 배포 전략 비교

전략	동작 방식	다운타임	롤백 속도	리스크

전략	동작 방식	다운타임	롤백 속도	리스크
Rolling	인스턴스를 순차적으로 새 버전으로 교체	없음	느림	중간 — 신/구 버전 공존
Blue/Green	새 환경(Green)을 구성 후 트래픽을 한 번에 전환	없음	빠름 (즉시)	낮음 — 즉시 롤백 가능
Canary	소량 트래픽(1~5%)에만 새 버전 적용 후 점진 확대	없음	빠름	매우 낮음
Recreate	기존 버전 중단 후 새 버전 배포	있음	느림	높음
A/B Testing	특정 사용자 그룹에만 새 버전 노출	없음	빠름	낮음

Blue/Green 배포 상세

배포는 3단계 페이즈로 진행된다.

페이즈	Load Balancer 트래픽 분배	설명
Phase 1	Blue v1.0 → 100%	현재 버전(Blue)이 운영 중, Green은 미배포 대기 상태
Phase 2	Blue v1.0 → 100% / Green v2.0 → 0% (테스트 중)	Green에 새 버전 배포 후 테스트 진행
Phase 3	Green v2.0 → 100% / Blue v1.0 → 0% (대기)	트래픽을 Green으로 전환 완료

롤백: Load Balancer를 다시 Blue로 전환 (수 초 내 완료)

Canary 배포 상세

새 버전을 소량의 트래픽에만 먼저 적용한 뒤 메트릭을 모니터링하며 점진적으로 확대한다.

단계	v1.0 트래픽	v2.0 트래픽	조건
Step 1	99%	1% (카나리)	에러율, 지연 등 메트릭 모니터링
Step 2	점진 감소	10% → 25% → 50% → 100%	이상 없을 경우 단계적 확대
이상 감지 시	100%	0%	즉시 롤백

Feature Flag (기능 플래그)

배포와 릴리스를 분리하는 기법. 코드는 배포하되, 기능 활성화는 플래그로 제어한다.

```
# Feature Flag 예시
if feature_flags.is_enabled("new_checkout_flow", user_id=user.id):
    return new_checkout()    # 새 기능
```

```
else:
    return legacy_checkout() # 기존 기능
```

- **장점**: 배포 위험 최소화, A/B 테스트, 점진적 롤아웃
- **대표 도구**: LaunchDarkly, Unleash, AWS AppConfig, Flipt

---

7. 브랜치 전략

주요 브랜치 전략 비교

전략	복잡도	배포 빈도	적합한 팀
GitHub Flow	낮음	높음 (수시 배포)	소규모, 빠른 반복
Git Flow	높음	낮음 (정기 릴리스)	대규모, 버전 관리 필요
Trunk-Based Development	낮음	매우 높음	CI/CD 성숙 팀
GitLab Flow	중간	중간	환경별 브랜치 필요 시

GitHub Flow (가장 심플)

`main` 브랜치는 항상 배포 가능한 상태를 유지하며, 모든 개발은 feature 브랜치에서 진행 후 PR을 통해 병합한다.

1. `main`에서 feature 브랜치 생성
2. feature 브랜치에서 개발
3. Pull Request 생성
4. 코드 리뷰 + CI 테스트 통과
5. `main`에 병합 → 자동 배포

흐름: main 브랜치 분기 → feature 브랜치 개발 → PR + 코드 리뷰 → CI 통과 → main 병합 → 자동 배포

Git Flow (복잡하지만 체계적)

브랜치	역할
`main`	릴리스 태그가 붙는 안정 브랜치
`develop`	개발 통합 브랜치. 모든 feature가 여기로 병합됨
`feature/*`	개별 기능 개발 브랜치. develop에서 분기 후 develop으로 병합
`release/*`	릴리스 준비 브랜치. develop에서 분기 후 main으로 병합
`hotfix/*`	프로덕션 긴급 수정. main에서 분기 후 main과 develop 양쪽으로 병합

흐름: develop 분기 → feature 개발 → develop 병합 → release 브랜치 준비 → main 병합 (릴리스 태그)

Trunk-Based Development (고성능 팀)

main 브랜치에 직접 커밋하거나, 수명이 1~2일 이내인 매우 짧은 브랜치를 사용한다.

- **특징**: main 브랜치에 직접 커밋하거나 매우 짧은 브랜치 사용
- **전제 조건**: 높은 테스트 커버리지, Feature Flag, 성숙한 CI/CD
- Google, Meta 등 대규모 기업에서 사용하는 방식

---

8. CI/CD 도구 비교

주요 CI/CD 도구

도구	유형	호스팅	설정 방식	특징
GitHub Actions	SaaS (내장)	GitHub 호스팅 러너 / 셀프호스팅	YAML	GitHub 네이티브 통합, Marketplace, 간편한 설정
Jenkins	오픈소스	셀프호스팅	Groovy (Jenkinsfile)	1800+ 플러그인, 높은 커스터마이징, 관리 부담 큼
GitLab CI	SaaS / 셀프호스팅	GitLab 러너	YAML	올인원 DevOps 플랫폼, 보안 스캔 내장, Auto DevOps
CircleCI	SaaS	클라우드 / 셀프호스팅	YAML	빠른 빌드, Docker 우수 지원
AWS CodePipeline	SaaS	AWS 관리형	JSON/콘솔	AWS 서비스 네이티브 통합
Tekton	오픈소스	Kubernetes 네이티브	YAML (CRD)	클라우드 네이티브, K8s 기반 파이프라인
Argo Workflows	오픈소스	Kubernetes 네이티브	YAML (CRD)	DAG 기반 워크플로우, ML 파이프라인에 적합

Jenkins vs GitHub Actions vs GitLab CI 심화 비교

항목	Jenkins	GitHub Actions	GitLab CI
설치/설정	수동 설치, 에이전트 구성 필요	설정 불필요 (GitHub 내장)	GitLab 내장 또는 셀프호스팅
학습 곡선	높음 (Groovy, 플러그인)	낮음 (YAML)	낮음 (YAML)
확장성	1800+ 플러그인	Marketplace Actions	내장 기능 풍부
보안 스캔	서드파티 플러그인 필요	서드파티 Actions	SAST/DAST/SCA 내장
비용	무료 (인프라 비용만)	무료 티어 2,000분/월 (Linux)	무료 티어 400분/월
VCS 연동	Git, SVN, Perforce 등 다양	GitHub 전용	GitLab 기본, 미러링 지원
UI	웹 대시보드	GitHub PR 내 통합	GitLab 내 통합
컨테이너 지원	Docker 플러그인	Docker 기본 지원	Docker/K8s 기본 지원

항목	Jenkins	GitHub Actions	GitLab CI
적합한 경우	레거시 인프라, 고도 커스터마이징	GitHub 사용 팀, 빠른 시작	올인원 DevOps, 규정 준수 필요

선택 가이드

조건	추천 도구
GitHub에 코드를 호스팅하는 경우	**GitHub Actions** (대부분의 경우 최적)
GitHub + 고도 커스터마이징 필요	GitHub Actions + **Jenkins** 병행 검토
GitLab을 사용하는 경우	**GitLab CI**
멀티 VCS 환경	**Jenkins**
AWS 서비스에 완전히 의존하는 경우	**AWS CodePipeline + CodeBuild**

---

9. GitOps

GitOps란?

Git을 Single Source of Truth로 사용하여 인프라와 애플리케이션의 상태를 관리하는 운영 방식.
기존 CI/CD가 "Push" 방식이라면, GitOps는 "Pull" 방식이다.

Push vs Pull 기반 배포

구분	Push 방식 (기존 CI/CD)	Pull 방식 (GitOps)
동작 방식	CI 서버가 kubectl apply로 클러스터에 직접 배포	클러스터 내 GitOps Agent가 Git Repo를 감시하며 자동 적용
권한 위치	CI 서버에 클러스터 접근 권한 필요	클러스터 외부에 접근 권한 불필요
보안	외부 시스템의 클러스터 접근 권한 노출 위험	클러스터 내부에서 풀링하므로 보안상 유리
주요 도구	Jenkins, GitHub Actions (kubectl 직접 실행)	ArgoCD, FluxCD

흐름 (Push): CI 서버 → kubectl apply → Kubernetes 클러스터

흐름 (Pull): Git Repo (원하는 상태) ← GitOps Agent 감시 → 자동 Reconcile → Kubernetes 클러스터

GitOps의 4가지 원칙

1. **선언적 (Declarative)**: 시스템의 원하는 상태를 선언적으로 기술
2. **버전 관리 (Versioned & Immutable)**: 모든 변경 이력이 Git에 남아 감사 추적 가능
3. **자동 풀 (Pulled Automatically)**: 에이전트가 자동으로 원하는 상태를 클러스터에 적용
4. **지속적 조정 (Continuously Reconciled)**: 드리프트(실제 상태 ≠ 원하는 상태) 자동 감지 및 복구

GitOps 도구: ArgoCD vs FluxCD

항목	ArgoCD	FluxCD
UI	풍부한 웹 UI (상태 시각화, 로그 조회)	UI 없음 (CLI + Grafana 연동)
아키텍처	별도 서비스로 동작	Kubernetes 네이티브 컨트롤러
Helm 지원	내장 렌더링	네이티브 Helm Controller
멀티 클러스터	중앙 집중식 관리	클러스터별 독립 설치
RBAC	자체 RBAC 시스템	Kubernetes 네이티브 RBAC
Self-Healing	지원 (자동 동기화)	지원 (지속적 조정)
적합한 경우	시각적 관리 필요, 중앙 제어	K8s 네이티브 지향, 가벼운 구성

GitOps 워크플로우 예시

1. 개발자가 앱 코드 변경 → GitHub PR → CI 파이프라인
2. CI에서 빌드 → 테스트 → Docker 이미지 빌드 → 이미지를 ECR에 Push
3. CI가 매니페스트 리포지토리의 이미지 태그를 업데이트 (자동 커밋)
4. ArgoCD/Flux가 매니페스트 리포지토리의 변경을 감지
5. 자동으로 Kubernetes 클러스터에 새 버전 배포
6. 드리프트 발생 시 Git 상태로 자동 복구

---

10. DevSecOps — 보안 통합

Shift-Left Security

보안을 개발 초기 단계로 당기는("왼쪽으로 이동") 접근법.
배포 후 보안 검사가 아닌, **코드 작성 시점부터** 보안을 파이프라인에 내장한다.

방식	흐름
기존 방식	Plan → Code → Build → Test → Deploy → **[Security]** → Operate (너무 늦음)
DevSecOps	Plan → **[Sec]** → Code → **[Sec]** → Build → **[Sec]** → Test → **[Sec]** → Deploy → **[Sec]** → Operate

각 단계의 보안 활동:

단계	보안 활동
Plan	위협 모델링
Code	코드 리뷰 + SAST
Build	이미지 스캔 + SCA
Test	DAST + 침투 테스트
Deploy/Operate	런타임 보안

CI/CD 파이프라인 보안 스캔 유형

유형	약어	설명	실행 시점	대표 도구
정적 분석	SAST	소스 코드의 보안 취약점 분석	빌드 시	SonarQube, Semgrep, CodeQL
소프트웨어 구성 분석	SCA	오픈소스 의존성의 알려진 취약점 검사	빌드 시	Snyk, Dependabot, OWASP Dep-Check
동적 분석	DAST	실행 중인 애플리케이션에 대한 공격 시뮬레이션	스테이징 배포 후	OWASP ZAP, Burp Suite
컨테이너 스캔	-	Docker 이미지의 취약점 검사	이미지 빌드 후	Trivy, Grype, Amazon ECR 스캔
시크릿 탐지	-	코드에 노출된 API 키, 비밀번호 등 검출	커밋 시 (pre-commit)	GitLeaks, TruffleHog, detect-secrets
IaC 보안 검사	-	CloudFormation, Terraform 설정의 보안 문제 검사	PR 시	Checkov, tfsec, CFN Guard

파이프라인 보안 체크리스트

- ☐ 시크릿은 코드에 하드코딩하지 않음 (Secrets Manager / Vault 사용)
- ☐ 모든 의존성에 대해 SCA 스캔 실행
- ☐ Docker 이미지는 최소 베이스 이미지 사용 (Alpine, Distroless)
- ☐ 이미지 빌드 후 취약점 스캔 통과 필수
- ☐ HTTPS/TLS 강제
- ☐ IAM 최소 권한 원칙 적용 (CI/CD 서비스 계정)
- ☐ 파이프라인 로그에서 민감 정보 마스킹
- ☐ 아티팩트 서명 및 무결성 검증

---

11. 컨테이너와 CI/CD

Docker 기반 CI/CD

컨테이너는 CI/CD에서 **환경 일관성** 문제를 해결하는 핵심 기술이다.

CI 파이프라인에서 Docker를 활용하는 방식:

1. Dockerfile로 애플리케이션 이미지 빌드
2. 이미지 내에서 테스트 실행
3. 이미지를 레지스트리(ECR, Docker Hub)에 Push
4. 동일한 이미지를 staging → production에 배포

→ "내 PC에서는 되는데" 문제 해결

멀티스테이지 빌드 (Multi-stage Build)

빌드 환경과 실행 환경을 분리하여 최종 이미지 크기를 최소화한다.

```dockerfile
# Stage 1: 빌드 환경
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: 실행 환경 (최소 이미지)
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

- **Stage 1 (builder)**: 전체 Node.js 환경에서 빌드
- **Stage 2 (runtime)**: Alpine 기반 경량 이미지에 빌드 결과만 복사
- **결과**: 이미지 크기 대폭 감소 (예: 1GB → 150MB)

Kubernetes에서의 CI/CD 흐름

Git Push 이후 전체 흐름:

단계	설명
1. CI 실행	GitHub Actions / Jenkins에서 코드 빌드 + 테스트
2. 이미지 빌드	Docker 이미지 빌드 후 ECR/Docker Hub에 Push
3. 매니페스트 업데이트	K8s 매니페스트의 이미지 태그 업데이트
4. GitOps 에이전트 적용	ArgoCD / Flux가 변경을 감지하여 클러스터에 롤링 업데이트 적용
5. 결과 처리	헬스 체크 + 모니터링 → 정상(완료) or 이상(자동 롤백)

흐름: Git Push → CI (빌드+테스트+이미지 Push) → 매니페스트 업데이트 → GitOps Agent → K8s 롤링 업데이트 → 헬스 체크 → 완료 or 자동 롤백

---

12. DORA 메트릭 — 성과 측정

DORA란?

DORA(DevOps Research and Assessment) 는 Google Cloud 소속 연구팀이 제시한 DevOps 성과 측정 프레임워크.
6년간의 연구를 통해 소프트웨어 팀의 성과를 나타내는 **4가지 핵심 메트릭**을 정의했다.

4가지 핵심 메트릭

메트릭	정의	측정 대상

메트릭	정의	측정 대상
배포 빈도 (Deployment Frequency)	프로덕션에 코드를 얼마나 자주 배포하는가	속도
변경 리드 타임 (Lead Time for Changes)	커밋 → 프로덕션 배포까지 걸리는 시간	속도
변경 실패율 (Change Failure Rate)	배포 후 장애/롤백이 발생하는 비율	안정성
서비스 복구 시간 (MTTR, Mean Time to Restore)	장애 발생 → 서비스 복구까지 걸리는 시간	안정성

성과 수준별 벤치마크

메트릭	Elite	High	Medium	Low
배포 빈도	하루 여러 번	주 1회 ~ 월 1회	월 1회 ~ 6개월 1회	6개월 이상
변경 리드 타임	1시간 미만	1일 ~ 1주	1주 ~ 1개월	1개월 이상
변경 실패율	0~5%	5~10%	10~15%	15% 이상
MTTR	1시간 미만	1일 미만	1일 ~ 1주	1주 이상

DORA 메트릭의 핵심 인사이트

측면	관련 메트릭	핵심 메시지
속도	배포 빈도, 변경 리드 타임	빠르게 배포할수록 좋은 팀
안정성	변경 실패율, MTTR	안정적으로 운영할수록 좋은 팀
핵심 인사이트	4가지 전체	속도와 안정성은 trade-off가 아님. 높은 성과 팀은 둘 다 동시에 달성한다

> **핵심**: DORA 연구에 따르면, 높은 성과를 내는 팀은 속도와 안정성을 **동시에** 달성한다. 빠른 배포와 높은 안정성은 상충하지 않는다.

DORA 메트릭 수집 방법

메트릭	데이터 소스
배포 빈도	CI/CD 파이프라인 로그 (성공한 프로덕션 배포 수)
변경 리드 타임	VCS 커밋 시간 ~ 배포 완료 시간 차이
변경 실패율	이슈 트래커(Jira, Linear)의 프로덕션 인시던트 / 총 배포 수
MTTR	인시던트 보고 시각 ~ 복구 완료 시각

---

13. CI/CD Best Practices (2026)

파이프라인 설계

1. **Pipeline as Code**: 파이프라인을 YAML/Groovy로 정의하고 Git에서 버전 관리
2. **Build Once, Deploy Anywhere**: 아티팩트를 한 번 빌드하고 모든 환경에 동일하게 배포
3. **Immutable Artifacts**: 빌드 결과물은 변경 불가. 수정 시 새로 빌드
4. **Fail-Fast**: 가장 빠른 테스트부터 실행하여 즉시 피드백
5. **병렬 실행**: 독립적인 작업은 병렬로 실행하여 전체 시간 단축
6. **캐싱**: 의존성, Docker 레이어 등을 캐싱하여 빌드 시간 단축

보안

7. **Shift-Left Security**: 보안 스캔을 파이프라인 초기 단계에 내장
8. **시크릿 관리**: 코드에 비밀정보 하드코딩 금지. Secrets Manager / Vault 사용
9. **최소 권한**: CI/CD 서비스 계정에 필요한 최소한의 권한만 부여
10. **아티팩트 서명**: 빌드 결과물의 무결성 보장

배포

11. **자동 롤백**: 헬스 체크 실패 시 자동으로 이전 버전으로 롤백
12. **Progressive Delivery**: Canary, Blue/Green 등 점진적 배포 전략 사용
13. **Feature Flag**: 배포와 릴리스를 분리하여 위험 최소화
14. **환경 패리티**: dev, staging, production 환경을 최대한 동일하게 유지

문화

15. **깨진 빌드 즉시 수정**: 빌드 실패 상태를 방치하지 않는 문화
16. **작은 단위 배포**: 대규모 변경보다 작은 변경을 자주 배포
17. **배포 추적성**: 모든 배포를 특정 커밋까지 추적 가능하게 유지
18. **피드백 루프**: 모니터링 데이터를 개발에 반영하는 순환 구조

2026 신규 트렌드

19. **AI 기반 파이프라인 최적화**: 테스트 우선순위 지정, 이상 탐지, 자동 롤백 판단
20. **Platform Engineering**: 내부 개발자 플랫폼(IDP)을 통해 CI/CD를 표준화/셀프서비스화

---

14. 실습 예제 — GitHub Actions 워크플로우

기본 구조

GitHub Actions 워크플로우는 `.github/workflows/` 디렉토리에 YAML 파일로 정의한다.

```yaml
# .github/workflows/ci.yml
name: CI Pipeline

# 트리거 조건
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

```yaml
# 환경 변수
env:
  NODE_VERSION: '20'

# 잡 정의
jobs:
  # ── 빌드 및 테스트 ──
  build-and-test:
    runs-on: ubuntu-latest   # GitHub 호스팅 러너

    steps:
      # 1. 코드 체크아웃
      - name: Checkout code
        uses: actions/checkout@v4

      # 2. Node.js 설정
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: 'npm'          # 의존성 캐싱

      # 3. 의존성 설치
      - name: Install dependencies
        run: npm ci

      # 4. 린트 검사
      - name: Run linter
        run: npm run lint

      # 5. 유닛 테스트
      - name: Run unit tests
        run: npm test -- --coverage

      # 6. 빌드
      - name: Build application
        run: npm run build

      # 7. 아티팩트 업로드
      - name: Upload build artifact
        uses: actions/upload-artifact@v4
        with:
          name: build-output
          path: dist/
```

Docker 빌드 + 배포 파이프라인

```yaml
# .github/workflows/cd.yml
name: CD Pipeline

on:
```

```yaml
  push:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  # ── CI: 빌드 + 테스트 ──
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm test

  # ── 보안 스캔 ──
  security-scan:
    runs-on: ubuntu-latest
    needs: test               # test 잡 성공 후 실행
    steps:
      - uses: actions/checkout@v4
      - name: Run Trivy vulnerability scan
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: 'fs'
          scan-ref: '.'

  # ── Docker 이미지 빌드 + Push ──
  build-and-push:
    runs-on: ubuntu-latest
    needs: [test, security-scan]  # 두 잡 모두 성공 후 실행
    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v4

      - name: Log in to Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
```

```yaml
          push: true
          tags: |
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${{ github.sha }}
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:latest

  # ── 스테이징 배포 ──
  deploy-staging:
    runs-on: ubuntu-latest
    needs: build-and-push
    environment: staging       # GitHub Environment (보호 규칙 설정 가능)

    steps:
      - name: Deploy to staging
        run: |
          echo "Deploying to staging environment..."
          # kubectl set image deployment/app app=${{ env.REGISTRY }}/${{
env.IMAGE_NAME }}:${{ github.sha }}

  # ── 프로덕션 배포 (수동 승인) ──
  deploy-production:
    runs-on: ubuntu-latest
    needs: deploy-staging
    environment: production    # 수동 승인 게이트 설정

    steps:
      - name: Deploy to production
        run: |
          echo "Deploying to production environment..."
          # kubectl set image deployment/app app=${{ env.REGISTRY }}/${{
env.IMAGE_NAME }}:${{ github.sha }}
```

워크플로우 실행 순서

순서	잡 이름	의존 관계	비고
1	`test`	-	가장 먼저 실행
2	`security-scan`	`test` 성공 후	
3	`build-and-push`	`test` + `security-scan` 모두 성공 후	
4	`deploy-staging`	`build-and-push` 성공 후	
5	`deploy-production`	`deploy-staging` 성공 후	GitHub Environment Protection Rule에 의한 수동 승인 필요

흐름: test → security-scan → build-and-push → deploy-staging → [수동 승인] → deploy-production

핵심 개념 정리

키워드	설명
`on`	트리거 이벤트 (push, pull_request, schedule, workflow_dispatch 등)
`jobs`	병렬로 실행되는 작업 단위
`needs`	잡 간 의존성 정의 (순차 실행)
`runs-on`	실행 환경 (ubuntu-latest, windows-latest, macos-latest, self-hosted)
`steps`	잡 내의 순차 실행 스텝
`uses`	재사용 가능한 Action 참조 (Marketplace)
`run`	쉘 명령어 직접 실행
`secrets`	GitHub에 저장된 비밀 변수 (API 키 등)
`environment`	배포 환경. 보호 규칙(수동 승인, 대기 시간 등) 설정 가능
`cache`	의존성 캐싱으로 빌드 시간 단축

---

15. 아키텍처 패턴 예시

패턴 1: 기본 CI/CD (GitHub Actions + AWS)

단계	도구 / 작업
소스	개발자 → GitHub (Push/PR)
CI	GitHub Actions: Lint + Unit Test → Build (npm/Docker) → Security Scan (Trivy) → Push to ECR
CD	AWS CodeDeploy: Blue/Green 배포 (EC2 / ECS) → 헬스 체크 + 자동 롤백
모니터링	CloudWatch: 모니터링 + 알람

흐름: 개발자 → GitHub → GitHub Actions (CI) → ECR → AWS CodeDeploy (Blue/Green) → CloudWatch

패턴 2: GitOps (GitHub Actions + ArgoCD + Kubernetes)

이 패턴에서는 App Repository(소스 코드)와 Config Repository(K8s 매니페스트)를 분리하여 관리한다.

단계	담당	작업
1. CI 트리거	App Repository	Push/PR 발생
2. CI 파이프라인	GitHub Actions	Test → Docker 이미지 빌드 → ECR Push
3. 매니페스트 업데이트	GitHub Actions	Config Repository의 이미지 태그 자동 커밋
4. GitOps 감지	ArgoCD	Config Repository 변경 감지 (Watch)
5. 클러스터 적용	Kubernetes	Rolling Update → Health Check → Auto Self-Heal

흐름: App Repo (Push) → GitHub Actions (CI + ECR Push) → Config Repo (이미지 태그 업데이트) → ArgoCD Watch → K8s 클러스터 배포

패턴 3: 풀스택 CI/CD (모노레포)

모노레포 구조에서 변경된 디렉토리에 따라 서로 다른 파이프라인이 동작한다.

변경 디렉토리	파이프라인	배포 대상
`/frontend`	npm test → npm build	S3 + CloudFront 배포
`/backend`	pytest → Docker build → ECR	ECS 배포
`/infra` (Terraform)	terraform plan (PR 코멘트)	terraform apply (main 병합 시)

흐름: Push → GitHub Actions → 변경 감지 → 해당 디렉토리별 파이프라인 실행

패턴 4: AWS 네이티브 CI/CD

AWS 서비스만으로 구성하는 완전 관리형 CI/CD 파이프라인.

스테이지	서비스	주요 작업
Source	CodeCommit 또는 GitHub 연동	소스 코드 가져오기
Build	CodeBuild	buildspec.yml 실행 → 유닛 테스트 → Docker 이미지 빌드 → ECR Push
Staging	CodeDeploy → ECS (스테이징)	스테이징 배포 + E2E 테스트
Approval	SNS 알림	수동 승인 대기
Production	CodeDeploy → ECS (프로덕션)	Blue/Green 배포 → CloudWatch 알람 → 자동 롤백

흐름: CodeCommit/GitHub → CodePipeline → CodeBuild → 스테이징 배포 → 수동 승인 → 프로덕션 Blue/Green 배포 → CloudWatch 자동 롤백

---

16. 참고 자료

리소스	URL
Red Hat — CI/CD 파이프라인 개념	https://www.redhat.com/ko/topics/devops/what-cicd-pipeline
GitHub Actions 공식 문서	https://docs.github.com/en/actions
Jenkins 공식 문서	https://www.jenkins.io/doc/
GitLab CI/CD 공식 문서	https://docs.gitlab.com/ci/
DORA Metrics — Google Cloud	https://dora.dev/
ArgoCD 공식 문서	https://argo-cd.readthedocs.io/

리소스	URL
FluxCD 공식 문서	https://fluxcd.io/docs/
Codefresh — DORA 메트릭 가이드	https://codefresh.io/learn/software-deployment/dora-metrics-4-key-metrics-for-improving-devops-performance/
KodeKloud — AWS DevOps Best Practices 2026	https://kodekloud.com/blog/aws-devops-best-practices-in-2026/
CI/CD Best Practices 2026	https://www.tekrecruiter.com/post/top-10-ci-cd-pipeline-best-practices-for-engineering-leaders-in-2026
Jenkins vs GitHub Actions vs GitLab CI 비교 (2026)	https://eitt.academy/knowledge-base/jenkins-vs-github-actions-vs-gitlab-ci-cicd-2026/
DevOps Engineering 2026 트렌드	https://www.refontelearning.com/blog/devops-engineering-in-2026-top-ci-cd-tools-trends-and-best-practices-github-actions-vs-jenkins

> 이 문서는 2026년 4월 기준으로 작성되었습니다. CI/CD 도구와 Best Practice는 빠르게 진화하므로, 각 도구의 공식 문서를 함께 참고하세요.